

Sortowanie

Naturalne sformułowanie problemu sortowania:

Dane wejściowe:

Lista obiektów $L = (a_1, \dots, a_n)$ ustalonego typu, w którym jest zdefiniowany liniowy porządek " $\leq$ ".

Dane wyjściowe:

Lista $L' = (b_1, \dots, b_n)$ będąca permutacją listy L taką, że $b_i \leq b_{i+1}$, dla $i = 1, \dots, n-1$.

Na ogół:

- elementy są strukturami (rekordami);
- o kolejności decyduje wartość wyróżnionego pola struktury: klucza;
- opisując algorytm najczęściej posługujemy się kluczami elementów, zapominając o pozostałej części elementu.

Sortowanie wewnętrzne/zewnętrzne

czyli w zależności od sposobu dostępu do danych:

1. Sortowanie wewnętrzne

czyli klasyczna wersja problemu sortowania

- lista reprezentowana za pomocą tablicy $A[0], \dots, A[n-1]$;
- pamięć wewnętrzna, dostęp bezpośredni;
- czas dostępu do elementu $A[i]$ stały, niezależny od rozmiaru danych n .

2. Sortowanie zewnętrzne

- lista jest plikiem w pamięci zewnętrznej;
- wykorzystuje się tylko stałą (możliwe że bardzo małą) ilość pamięci wewnętrznej;
- podstawowa własność pliku: dostęp sekwencyjny do kolejnych elementów, czyli:
 - dostęp do i -tego elementu wymaga przeglądnięcia poprzedzających go $i-1$ elementów.

Uwaga:

od tego miejsca, aż do dowołania, będziemy mówić o sortowaniu wewnętrznym.

Oznaczenia w zapisie algorytmów sortowania

- sortowana jest tablica $A[0..n-1]$ lub jej fragment $A[L..R]$,
- porównania wykonywane są operatorem $\leq$ lub $<$,
- funkcja $\text{swap}(a,b)$ zamienia elementy a i b w czasie $O(1)$.

Sortowanie tablicy indeksów

Wejściowe do sortowania:

$S[0..n-1]$ – tablica rekordów (struktur itp.)

$A[0..n-1]$ – tablica indeksów, $A[i] = i, i=0, \dots, n-1$

Wyjściowe:

tablica S bez zmian; tablica A posortowana, tzn.

$$S[A[0]] \leq S[A[1]] \leq \dots \leq S[A[n-1]]$$

Ma to sens gdy sortujemy duże struktury i nie chcemy ich w trakcie sortowania przestawiać.

Problem:

po sortowaniu tablicy indeksów A uporządkować tablicę rekordów S:

$$S[0] \leq S[1] \leq \dots \leq S[n-1]$$

Wymagania: czas liniowy, pamięć robocza stała.

Metoda: po cyklach permutacji. Rozwiązanie: ćwiczenie

Sortowanie stabilne**Definicja:**

Algorytm sortowania jest stabilny, jeśli zachowuje względną kolejność elementów o jednakowych kluczach.

Np. sortowanie rekordów składających się z 2 liczb.

Dane:

(3, 1) (2, 1) (3, 4) (1, 3) (3, 2) (2, 4) (1, 1)

- kluczem jest pierwsza liczba każdego rekordu,
- klucze się powtarzają, ale wszystkie rekordy są różne.

Możliwe są różne poprawne wyniki sortowania, np.

(1, 1) (1, 3) (2, 1) (2, 4) (3, 2) (3, 4) (3, 1)

Pary (1,1) i (1,3) są w innej kolejności niż na wejściu.

Sortowanie stabilne daje jednoznaczny wynik:

(1, 3) (1, 1) (2, 1) (2, 4) (3, 1) (3, 4) (3, 2)

Uwaga.

Każdy algorytm sortujący przez porównania można "ustabilnić", powiększając klucz o początkowy indeks. Zwiększa to oczywiście ogólny koszt algorytmu, czego chcemy uniknąć.

Proste (kwadratowe) metody sortowania

Sortowanie bąbelkowe (Bubblesort)

```
Bubblesort (A, n)
  for (k=n-1; k>0; k--)    // k = prawa granica
    for (j=0; j<k; j++)
      if (A[j+1]>A[j])
        swap(A[j], A[j+1]);
end.
```

Złożoność:

Operacje dominujące: porównania w instrukcji if.

$$(n-1) + (n-2) + \dots + 1 = n*(n-1)/2 .$$

Zatem pesymistyczna złożoność czasowa wynosi

$$T(n) = 1/2 * n^2 - 1/2 * n = \Theta(n^2)$$

Złożoność średnia: identyczna jak pesymistyczna - liczba porównań pesymistycznie i średnio ok. $1/2n^2$.

Liczba zamian (swap) średnio ok. $1/4n^2$.

Fakt.

Bubblesort jest stabilny.

Innych oczywistych zalet brak.

Jest jednak pewna zaleta: "oblivious" (niepomny) - dla ustalonego n , niezależnie od wyników, w każdym kroku wykonuje zawsze porównanie tych samych komórek pamięci, dzięki temu algorytm ma swoją wersję równoległą w modelu sieci sortujących.

Możliwe są modyfikacje usprawniające w pewnych przypadkach, np.

- zapamiętanie miejsca ostatnio wykonanej zamiany,
- w kolejnej iteracji pętli zewnętrznej ustawiać górną granicę pętli wewnętrznej na zapamiętane miejsce,
- wtedy dla ciągu już na wejściu posortowanego złożoność będzie $O(n)$,

nie mają one jednak wielkiego znaczenia.

W szczególności, istnieją permutacje zawierające $O(n)$ inwersji dla których czas usprawnionej wersji będzie kwadratowy. Przykład?

Algorytm prostego wyboru (Selectionsort)

Idea:

- w i -tym kroku znajdź najmniejszy element spośród $A[i], \dots, A[n-1]$ i zamień go z elementem $A[i]$;
- wykonuj ten krok dla $i=0, \dots, n-2$

```
Selectionsort (A, n)
  for (i=0; i<n-1, i++)
    min = i;
    for (j=i+1; j<n; j++)
      if (A[j] < A[min])
        min = j;
    swap (A[i], A[min]);
end.
```

Analiza identyczna jak dla metody bąbelkowej:

$$T(n) = 1/2 * n^2 - 1/2 * n = \Theta(n^2)$$

Czyli: zarówno w pesymistycznym, jak i średnim przypadku, liczba porównań wynosi ok. $1/2 n^2$.

Najważniejsza zaleta:

wykonuje tylko $n-1$ zamian ($O(n)$ podstawień) – zatem jest to metoda zalecana do sortowania niedużych tablic zawierających duże struktury.

Wada: Selectionsort nie jest stabilny.

Przykład? [2, 2, 1].

Można go zmodyfikować ("ustabilnić") :

- zamiast zamieniać $A[i]$ z $A[\min]$ przesuwając $A[i..\min-1]$ w prawo o 1 i wstawiać $A[\min]$ w $A[i]$,
- ale wtedy algorytm wykonuje $\Omega(n^2)$ podstawień i traci swą najważniejszą zaletę,
- **więc wynik tej modyfikacji nie jest już algorytmem sortowania metodą (prostego) wyboru !**

Algorytm prostego wstawiania (Insertionsort),

- niezmiennik: $A[0..i-1]$ już jest uporządkowana;
- w i -tym kroku ($i=1,2,\dots$) wstawiamy $A[i]$ na właściwe miejsce w uporządkowanej $A[0..i-1]$;
 - aby to miejsce zrobić przesuwamy elementy większe od $A[i]$ leżące w $A[0..i-1]$ w prawo o 1 pozycję;
 - uzyskujemy w ten sposób posortowaną $A[0..i]$.

```
insertionsort (A, n)
  for (i=1; i<n; i++)
    tmp=A[i]; j=i-1;
    while (j>=0 and tmp < A[j])
      A[j+1]=A[j]; j--;
    A[j+1]=tmp;
end.
```

Złożoność pesymistyczna (ciąg posortowany malejąco na wejściu) identyczna jak dla poprzednich:

- pesymistycznie liczba porównań ok. $1/2 n^2$;
- średnia liczba porównań ok. $1/4 n^2$.

Zalety:

1. **stabilny**,
2. średnio dwukrotnie szybszy niż inne proste metody,
3. **(!!!) optymalny** dla ciągów prawie posortowanych.

Definicja:

- *ciąg n -elementowy jest prawie posortowany, jeśli liczba inwersji w tym ciągu jest liniowa ($O(n)$),*
- *formalnie: należy do pewnej klasy ciągów, w których inwersji jest nie więcej niż $c \cdot n$, dla pewnej stałej c .*

Algorytm prostego wstawiania wykonuje:

- porównań tyle ile jest inwersji w permutacji wejściowej plus $n-1$,
- podstawień o $n-1$ więcej niż porównań;
- zatem dla ciągu prawie posortowanego razem $O(n)$ operacji.

Wśród metod kwadratowych ma najwięcej zalet.

Modyfikacja, ważna teoretycznie:

Insertionsort + Binsearch

miejsce do wstawienia $A[i]$ ustalane metodą binarną:

- liczba przesunięć pozostaje $\Theta(n^2)$;
- liczba porównań zmniejsza się do około $n \lg n$ (bardzo blisko teoretycznej granicy dolnej);

Istniała hipoteza (już obalona), że ta metoda jest optymalna pod względem liczby wykonywanych porównań.

Sortowanie metodą scalania (mergesort)

klasyczny przykład algorytmu skonstruowanego metodą dziel i zwyciężaj.

0. Jeśli $n=1$ to nic nie rób (tablica jest posortowana),
w przeciwnym przypadku:
 1. Podziel tablicę na dwie części, w przybliżeniu jednakowej długości, lewą i prawą;
 2. Posortuj każdą część tą samą metodą (rekurencja);
 3. Scal otrzymane posortowane ciągi w jeden posortowany ciąg.

Scalanie wymaga użycia dodatkowej roboczej tablicy $B[]$ wielkości tej samej co tablica $A[]$.

```
Merge(A, L, m, R, B)
    static int B[n];
    for (i=L; i<=R; i++)
        B[i]=A[i];                // kopiuj A do B
    i=L; j=m+1; k=L;
    while (i<=m and j<=R)
        if (B[i]<=B[j])
            A[k++]=B[i++];
        else
            A[k++]=B[j++];
    // przepisz pozostałość: działa tylko jedna pętla
    while (i<=m) A[k++]=B[i++];
    while (j<=R) A[k++]=B[j++];
end;
```



```

Mergesort (A, L, R)           // sortuj A[L..R]
    if (L==R) return;
    m =  $\lfloor (L+R)/2 \rfloor$ ;
    Mergesort (A, L, m);
    Mergesort (A, m+1, R);
    Merge (A, L, m, R, B);
end;

```

Main:

```
Mergesort(A, 0, n-1);
```

Przećwicz na przykładzie:

Tablica A[8]:

indeksy:	0	1	2	3	4	5	6	7
wartości:	4	8	10	15	3	10	18	19

Ćwiczenie dla mergesort-u, w kontekście dziel i zwyciężaj:

- *narysuj drzewo wywołań rekurencyjnych,*
- *wpisz dane wejściowe i wyjściowe (zakresy) w każdym węźle,*
- *ponumeruj chronologicznie wywołania procedury mergesort i osobno wywołania procedury merge,*
- *jakim porządkom w drzewie wywołań odpowiadają numeracje wywołań?*
- *zauważ analogię między wywołaniami funkcji w algorytmie mergesort i algorytmami przeglądu drzewa binarnego.*

Analiza algorytmu sortowania metodą scalania.

Równanie rekurencyjne opisujące złożoność ma postać:

$$T(n) = \begin{cases} 1, & \text{jeśli } n = 1, \\ 2 T(n/2) + \Theta(n), & \text{jeśli } n > 1 \end{cases}$$

bo sumujemy koszty trzech kroków metody "dziel i zwyciężaj":

- koszt podziału wynosi $O(1)$;
- zawsze są dwa podzadania wielkości $n/2$;
- koszt scalania jest liniowy: $\Theta(n)$.

Z twierdzenia o rekurencji uniwersalnej:

$$T(n) = aT(n/b) + n^p, \quad n > 0,$$

$a=2$, $b=2$, $p=1$, przypadek 2:

$$T(n) = \Theta(n \log n)$$

Oszacujemy przybliżoną liczbę wykonywanych porównań kluczy

Założmy:

w algorytmie scalania dwóch ciągów posortowanych o sumarycznej długości n pesymistycznie wykonuje się n porównań.

Faktycznie wykonuje się nie więcej niż $n-1$ porównań, ale każdy element jest przepisywany, więc liczba iteracji wszystkich pętli razem wynosi dokładnie n .

Równanie na liczbę $T(n)$ wykonanych porównań:

$$T(n) = \begin{cases} 0, & \text{jeśli } n = 1, \\ 2 T(n/2) + n, & \text{jeśli } n > 1 \end{cases}$$

(pominęliśmy sytuacje gdy n nieparzyste)

Iterujemy, dla n będących potęgami dwójki:

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &= 2(2T(n/4) + n/2) + n \\ &= 2(2(2T(n/8) + n/4) + n/2) + n \\ &= 2(2(\dots 2(2(2T(1)+2) +4) + \dots + n/4) + n/2) + n \\ &= 2^{\lg n} T(1) + n \lg n = n \lg n \end{aligned}$$

Dla innych wartości n należy dodać składnik $O(n)$.

Fakt.

Liczba porównań wykonywanych w algorytmie Mergesort jest tylko o wartość $O(n)$ większa od dolnego ograniczenia na liczbę porównań w modelu algorytmów sortujących przez porównania (drzewa decyzyjne sortujące).

Druga ważna zaleta: **Mergesort jest stabilny**

(istotna jest nieostra nierówność w głównej pętli procedury scalania).

Jedyna wada, bardzo poważna:

złożoność pamięciowa (pamięć robocza): $\Theta(n)$

Mergesort praktycznie

- bez rekurencji, bez stosu, bottom-up;
tak jak sortowanie plików, czyli sortowanie zewnętrzne
- tablice A i B na kolejnych poziomach zmieniają się rolami;
- przebiegamy tablicę scalając pary sąsiednich ciągów posortowanych długości k , wynik wpisując do drugiej tablicy;
- kolejny przebieg w drugiej tablicy scala pary sąsiednich ciągów długości $2k$, itd.

- elegancki kod powstaje gdy skalamy ciągi bitoniczne:
 - drugi z pary skalanych sąsiednich ciągów jest zapisany w odwróconej kolejności (malejąco);
 - kod scalania bardzo się upraszcza, np. jeśli oba zajmują tablicę $a[0..n-1]$ to:

```
for (i=0,j=n-1,k=0; k<n; k++)
    if (a[i]<=a[j]) b[k++]=a[i++];
    else           b[k++]=a[j--];
```

Drobny problem ze scalaniem ciągu bitonicznego:

- psuje się stabilność w pewnym szczególnym przypadku:
 - gdy największy element danego fragmentu tablicy występuje w prawej połowie w więcej niż jednym egzemplarzu;
- można to naprawić najpierw (krótką pętlą) odwracając kolejność największych elementów z prawej połowy.